When “forget this data” breaks the model

A plain-language overview of a research project on machine unlearning

Khalil Bibi
CentraleSupélec

October 2026 — personal research, not peer reviewed

In one paragraph

Sometimes a trained AI model has to “forget” some of the data it learned from, for example when people ask for their data to be deleted. Retraining from scratch is expensive, so shortcuts are used. I studied one popular shortcut, **NegGrad+**, and found that it becomes unreliable when the data to forget all belongs to **the same category** (for example, 600 photos of boots), whereas the same amount of data spread across categories causes no trouble. Fixes I tested reduced the damage to the model, but mostly by forgetting less. The practical lesson: **test unlearning methods on deletion requests concentrated in one category** before trusting them.

1. The problem: making a model forget

A machine-learning model learns from examples. Imagine an image classifier trained on 55,000 photos of clothes (the Fashion-MNIST dataset, ten categories: t-shirts, trousers, boots, . . .). Later, some of these photos must be removed, for instance to honour a deletion request under privacy rules. The model has already absorbed them.

- The **gold standard** is to retrain the model from scratch without those photos. It is exact, but slow and costly for large models.
- **Approximate unlearning** methods instead adjust the existing model for a short time so that it “forgets” the photos while keeping its skills on everything else.

NegGrad+ is one such method. It does two things at once: it keeps practising on the data that stays, and it pushes the model to become *worse* on the data to forget. The second part is the delicate one: there is no natural limit to “getting worse”, so the push can run away.

2. The question

Deletion requests are rarely random. A request might come from one user, one product line or one type of image, so the data to forget can be **concentrated in one category**. I asked a narrow question:

With the same number of examples to forget (600 images), does it matter whether they all come from one category (“concentrated”) or are spread evenly across the ten categories (“spread”)?

In the concentrated case, the other examples of that category stay in the model's training data: the model should forget *these* 600 boots, not boots in general.

3. What happened

An unexpected failure

The first experiment looked like a success for NegGrad+, but for the wrong reason: in 26 of 55 runs the model's accuracy fell to 20% or less, always with concentrated requests. A model that no longer recognises anything trivially "forgets". This is how the project started: not with an answer, but with a warning sign.

Finding 1: concentration makes NegGrad+ unstable

Across follow-up studies, concentrated requests repeatedly sent NegGrad+ off the rails: its loss on the data to forget exploded (the training "diverged"), while spread requests of the same size never did. In one study with 14 independent repetitions, NegGrad+ diverged in 84 of 140 concentrated runs.

Finding 2: it is not just that some categories are harder

A natural objection: maybe some categories are simply difficult, and concentrated requests happen to pick them. To separate the two explanations, I tested 68 mixes of one, two, five or ten categories and used a statistical model that accounts for the difficulty of each category and for many ways of combining them. Even after this, **the more concentrated the request, the more the forgetting loss climbed in the first steps**, beyond what the categories themselves explain.

Finding 3: it is specific to how NegGrad+ forgets

I compared NegGrad+ with a "control" method that, instead of pushing the model to be wrong, teaches it random wrong labels for the data to forget (a target with a natural limit). Under NegGrad+, concentration made the training signals (the typical size of the gradients, before the safety clipping that caps them) about five times larger; under the control, only about 13% larger.

A likely suspect: BatchNorm

Most modern networks contain **BatchNorm** layers, which rescale their internal signals using the average of the current batch of images. When the batch to forget contains only boots, that average is very unusual. This seems to play a role, but the experiments do **not** prove that it is the mechanism: every fix I tested changed several things at once.

4. Can the problem be avoided?

I tried three kinds of fixes, each in a study whose analysis was fixed in advance (Section 6).

1. **Changing how BatchNorm treats the batch to forget.** Some variants avoided the problem in some settings but failed in others: no single tweak was a general cure.

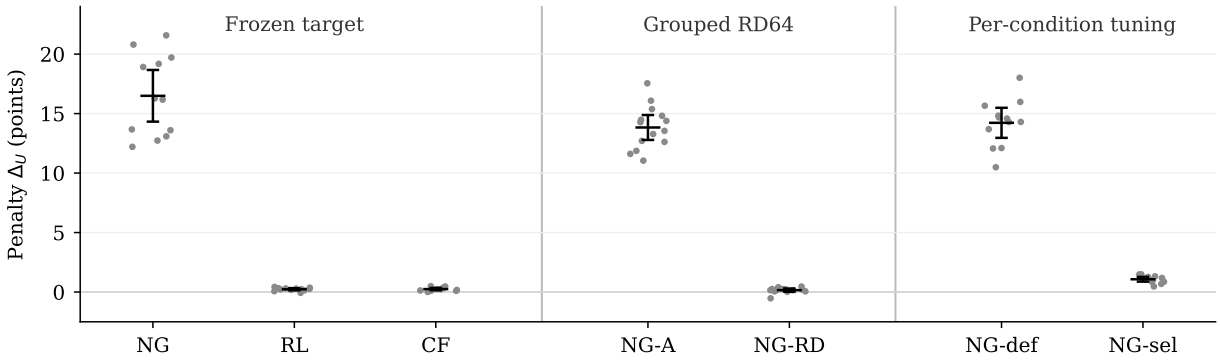


Figure 1: How much more accuracy on the remaining data the model loses with concentrated requests than with spread ones ("penalty", in percentage points). Each grey dot is one independent repetition; black marks show the average and its uncertainty. Each panel is a separate study, so compare bars only within a panel. NG: NegGrad+; RL: random-label control; CF: fixed alternative target; NG-A / NG-RD: NegGrad+ without / with the RD64 fix; NG-def / NG-sel: default / tuned settings.

2. **A combined BatchNorm fix ("RD64").** Normalising the batch to forget with statistics borrowed from the batch to keep removed almost all of the penalty (13.7 points less) and all divergences. But it changed three things at once, and it barely changed the model's predictions on the data to forget.
3. **Tuning the settings separately for each category.** With a grid of 20 settings, the penalty dropped from about 14 points to about 1 point. However, for 7 of the 10 categories, **no setting reached the forgetting strength of a model retrained from scratch without breaking**. Tuning "won" mostly by forgetting less.

A different forgetting method, which steers the model towards a fixed, sensible alternative answer, also preserved the model well, but it too barely changed its predictions on the data to forget.

A measurement trap worth knowing

Many runs where the model looked "broken" (accuracy at 20% or less) were not really broken. The damage was in BatchNorm's stored statistics, which were corrupted by the batches to forget. After recomputing those statistics on the remaining data, without touching anything else, almost all of these models recovered to about 70–74% accuracy (the original model scores about 93%). A real loss remained, but much smaller than it first appeared. How you evaluate an unlearned model can change the conclusion.

5. What this means in practice

If you deploy an unlearning method like NegGrad+ in a network with BatchNorm:

1. **Look at what is being deleted.** Requests concentrated in one category are the risky case.
2. **Test on concentrated requests** before trusting a method. Evenly spread test cases hid the failure completely.
3. **Measure several things separately**: whether training blew up, whether the model still works, how much accuracy it lost, and how much it actually changed on the deleted data. Looking at one number alone can mislead in both directions.

4. **Do not assume tuning will save you**: it mainly made the method act more weakly.

What this project does *not* show

- It does **not** show that any method truly erased the data or protects privacy: none of the studies measured that.
- It does **not** identify the exact cause; BatchNorm is a suspect, not a proven mechanism.
- It does **not** prove that the results hold for every model or dataset: most experiments use one small network on Fashion-MNIST, with smaller checks on CIFAR-10 and Tiny ImageNet.

6. How the work was done

To avoid fooling myself, each of the later studies followed the same routine before any result was seen:

1. write down the question, the single main test and the wording of the possible conclusions;
2. simulate the experiment to choose how many independent repetitions were needed and to check that the test does not give false alarms;
3. save and timestamp the plan and the code in a version-controlled repository;
4. run a small pilot only to measure computing time;
5. run the study on a cloud GPU, then apply the analysis exactly as written.

Independent repetitions use different random seeds (a new model each time), and the statistics treat each repetition as one data point. The studies share some data, so they are not fully independent replications, and the plans were stored in a private repository rather than publicly registered.

A detailed technical working paper with all studies, numbers and appendices is available on request.